Оператор while

while вираз_умови_повтору_test:
блок_операторів

Блок операторів циклу while повторюється до того часу, поки результатом логічного висловлювання умови повтору є значення True. Вираз умови обчислюється перед кожним повтором циклу, і вихід з циклу (тобто перехід до оператора, наступного за оператором while) здійснюється в тому випадку, якщо вираз умови має значення False.

Приклад Виведемо значення від 0 до 4 на екрані=0

```
i=0
while i < 5:
    print("значення лічильника=", i)
    i=i+1 # i+=1
print("end")
```

Задається значення змінної i=0, перевіряється умова i<5, так як вона виконується, то переходимо до операторів в середині циклу(виводимо значення і на екран та збільшуємо на 1, тобто i=0+1=1). Знову повертаємось до перевірки умови на початку i<5, вона виконується -> виконуються оператори виводу на екран та збільшення і, і так до тих пір, поки значення і не стане дорівнювати 5. Тоді оператори всередині циклу вже не будуть виконуватись, виконання перейде до оператора print("end")

```
while вираз_умови_повтору_test:
    блок_операторів1
    if умова1 : break# вийти з циклу пропустивши else до оператор4
    if умова2 :continue# перейти на початок циклу до виразу_умови_test
    блок_операторів6
else: оператори3# виконуються якщо вираз_умова_повтору  неправда
оператор4
```

В деяких випадках оператор while можна представити наступною схемою

```
while (а логічний_оператор b):
    оператори
    зміна а або зміна b
```

Приклад
```
X=input(“Двері відчинені?”)
while X!=”так”:
        print("стукаємо")
        X=input(“Двері відчинені?”)
print("Заходимо")
```

Блок операторів у while може не виконатись жодного разу, якщо вираз умова прийме значення False.

Приклад Виведемо на екран значення функції sin для x, починаючи з нуля з кроком 0,1, доки синус не почне приймати негативні значення.x=0
```
import math
x=0
while math.sin(x)>=0:
        print("sin({0})={1}".format(x,math.sin(x)))
        x=x+0.1
```

Приклад. Будемо просити користувача ввести позитивне число, доки він його не введе
```
        x=int(input(“Введіть позитивне x=”))
        while x<=0:
                x=int(input(“Введіть позитивне x=”))
        print(“Дякую”)
```

Приклад. Знайдемо суму чисел, які користувач вводить з клавіатури, введення закінчиться, коли введе 0
```
s=0
x=float(input())
while x<>0:
        s=s+x# s+=x
        x=float(input())
print(“summa=”,s)
```
Як змінити код, щоб порахувати число парних та непарних.

Приклад. Знайти суму ряду видів, що є розкладанням функції з заданною точністю exp(x)=1+x+x^2/2!+...+x^n/n!+.. (x^i/i!)
```
import math
s=0
i=0
```

```
x=1
eps=0.000000001
while ((x**i/math.factorial(i))>eps):
    s=s+x**i/math.factorial(i)
    i+=1
print(s) # 2.7182539682539684
print(i)# 8
```

Будь-який цикл for можна записати за допомогою while, зворотне взагалі кажучи невірно.

Інструкція pass - не виконує жодних дій і використовується у випадках, коли синтаксис мови потребує наявності інструкції, але поки що жодних корисних дій виконати не потрібно. Може бути використана як порожнє тіло циклу чи умовного оператора.

if True: pass# інструкції будуть написані пізніше

while True:

pass# інструкції будуть написані пізніше

Проблема "зациклювання"

Іноді при неуважному використанні оператора while виникає так зване зациклювання - нескінченне повторення тіла циклу. Це може статися з двох причин.

1. Змінні, що є частиною висловлювання умови циклу, не приймають всередині циклу коректних значень, у результаті результатом висловлювання умови завжди буде значення True.

2. Некоректним є сам вираз умови циклу.

Приклад.

```
while True:
    print("Натисні Ctrl+c")
```

Приклад.

```
x=0
while x!=1:
        print(x);
         x=x+0.3
```

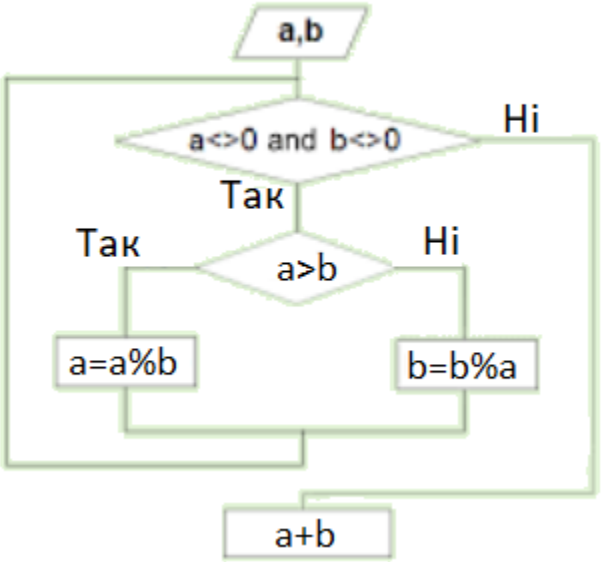
Приклад. Знайдемо середнє арифметичне простих чисел, доки користувач не введе 0

```
x = int(input()) # Вводимо перший елемент
k = 0
S = 0
while x != 0:
    # Перевіримо x на простоту
    f=True
    for i in range (2, x//2+1):
        if x % i == 0: # Якщо x - не просте
          f=False
    if f:
        k += 1
        S += x
    # Вводимо нове x
    x = int(input())
if k == 0:
    print('Простих чисел немає')
else:
    print('Середнє арифметичне простих:', S/k)
```

Приклад. Дано натуральне число N. Чи вірно, що в ньому немає цифри 4?

```
N = int(input('N = '))
while N > 0:
   x = N % 10
   if x == 4:
      print('Є 4')
      break
   N = N // 10
else:
   print('Немає 4')
```

Завдання. За допомогою блок схеми реалізувати Алгоритм Евкліда пошуку найбільшого спільного дільника



Приклад. Вивести числа від 1 до 10

```
i = 1
while i <= 10:
    print(i)
    i += 1
```

Завдання на парі

1. ***Вивести парні числа від 2 до 20***
2. ***Порахувати суму чисел від 1 до заданого n (введення з клавіатури)***
3. ***Запитувати у користувача число, поки він не введе 0***
5. ***Вивести всі цифри числа по одній (наприклад, для 123 → 3, 2, 1)***
6. ***Вивести кількість цифр у введеному числі***
7. ***Вивести факторіал числа n (n! = 1×2×…×n)***
8. ***Гра: вгадати число***
    - ***Програма загадує число від 1 до 10.***
    - ***Користувач вводить варіанти, поки не вгадає.***
9. ***Перевірити, чи є введене число простим (ділиться тільки на 1 і на себе)***
10. ***Вивести всі числа Фібоначчі, менші за n***